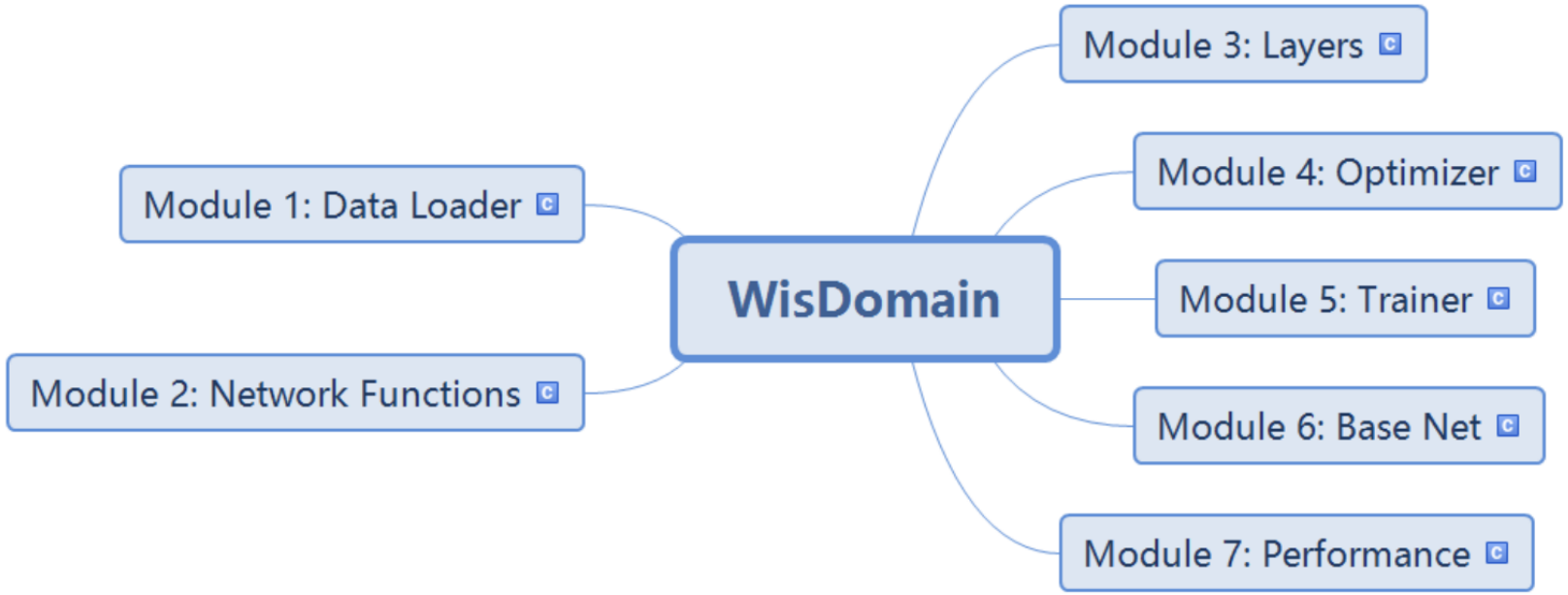
DESIGN YOUR OWN DEEP LEARNING FRAMEWORK

XU Chentianye, Machine Learning Association

总体设计


WisDomain
Module 1: Data Loader
Module 2: Network Functions
Module 3: Layers
Module 4: Optimizer
Module 5: Trainer
Module 6: Base Net
Module 7: Performance

各模块功能与实现

一、数据加载模块（Data Loader)

该模块主要用于对指定的数据集进行下载、加工与处理等操作，根据用户传入的参数获悉用户需要哪个数据集，且需要该数据集被处理成何种形式，返回经过不同处理的数据集以便用于后续的网络学习。

各模块功能与实现

二、相关函数模块（Network Functions）

该模块定义了重要的激活函数及其导数，以及损失函数等等，例如Sigmoid函数及其导数、ReLU函数及其导数、Softmax函数、均方误差函数、交叉熵函数、Softmax损失函数等等，为后续相关网络层的定义和实现，以及网络模型的定义与实现提供了基础。

为了提升神经网络的速度，这些函数中有些函数使用numba库进行了加速。

numba支持的numpy方法有限，如果对包含不支持方法的函数使用了装饰器，则会报错或警告。

对照官网提供的支持方法与属性的说明，我们将一些函数进行改写，以用支持的方法和属性代替原来的实现，再使用numba装饰器。

各模块功能与实现

Calculation

The following methods of Numpy arrays are supported in their basic form (without any optional arguments):

- `all()`
- `any()`
- `argmax()`
- `argmin()`
- `conj()`
- `conjugate()`
- `cumprod()`
- `cumsum()`
- `max()`
- `mean()`
- `min()`
- `nonzero()`
- `prod()`
- `std()`
- `take()`
- `var()`

各模块功能与实现

三、网络层模块（Layers）

该模块在基础网络层Base Layer类的基础上继承定义了多种网络层类，包括仿射层、卷积层、图卷积层等等，这些层都包含权重和偏置，网络反向传播更新时实际上是这些层的权重和偏置进行更新，它们也都有前向计算与反向传播方法。同时，为了以统一的接口进行前向计算与反向传播，并方便用户更清晰地根据需要构造神经网络模型，我们将激活函数以及代价函数也设计为网络层，如ReLU层、Sigmoid层、Softmax损失层等等，这些函数层，包括池化层，仅仅是提供计算服务，都不包含权重和偏置，反向传播时没有权重和偏置的更新。

各模块功能与实现

三、网络层模块（Layers）

该模块在基础网络层Base Layer类的基础上继承定义了多种网络层类，包括仿射层、卷积层、图卷积层等等，这些层都包含权重和偏置，网络反向传播更新时实际上是这些层的权重和偏置进行更新，它们也都有前向计算与反向传播方法。同时，为了以统一的接口进行前向计算与反向传播，并方便用户更清晰地根据需要构造神经网络模型，我们将激活函数以及代价函数也设计为网络层，如ReLU层、Sigmoid层、Softmax损失层等等，这些函数层，包括池化层，仅仅是提供计算服务，都不包含权重和偏置，反向传播时没有权重和偏置的更新。

各模块功能与实现

三、网络层模块（Layers）

```python
class BaseLayer:
    def __init__(self, W, b, stride, padding):
        self.W = W  # 权重
        self.b = b  # 偏置
        self.stride = stride  # 步长
        self.padding = padding  # 填充
        self.x = None  # 本层输入 or 掩码（ReLU层）
        self.a = None  # 本层输出
        self.xshape = None  # 输入的形状
        self.gradW = None  # 权重的梯度
        self.gradb = None  # 偏置的梯度
    def forward(self, x):
        # x 的第一维是样本个数
        self.xshape = x.shape
        x = x.reshape(x.shape[0], -1)  # 如果前一层是卷积，将张量拉成向量
        self.x = x
        a = np.dot(self.x, self.W) + self.b
        return a
    def backward(self, dadxProductNext):
        dadxProduct = np.dot(dadxProductNext, self.W.T)
        self.gradW = np.dot(self.x.T, dadxProductNext)
        self.gradb = np.sum(dadxProductNext, axis=0)
        dadxProduct = dadxProduct.reshape(self.xshape)  # 如果前一层是卷积，还原为原始张量形状
        return dadxProduct
```

各模块功能与实现

三、网络层模块（Layers）

反向传播的实现

以一层为例，其前向计算可以用下式表示：$\boldsymbol{a}_{n+1} = f(\boldsymbol{a}_n)$，其中 $f(\bullet)$ 是本层对输入施加的操作。

由于将激活函数单独设置为独立的层，这里就不需要在实际的层（如卷积层、全连接层）中考虑激活函数的作用，而是将它们的变换操作视为与激活函数同样地位的函数，这样我们就把卷积层、全连接层等等以及各种激活函数一起视为“功能层”，只是考虑其变换操作，而忽略其在网络模型中的角色。

使得反向传播的实现变得清晰明了，且能够定义高度统一的接口。

各模块功能与实现

三、网络层模块（Layers）

反向传播的实现

基于这种思想，则可以根据链式法则，将本层的权重（如果有）的梯度转化为：

$$\frac{\partial J}{\partial W_n} = \boxed{\frac{\partial J}{\partial \boldsymbol{a}_N} \cdot \frac{\partial \boldsymbol{a}_N}{\partial \boldsymbol{a}_{N-1}} \cdot \ldots \cdot \frac{\partial \boldsymbol{a}_{n+2}}{\partial \boldsymbol{a}_{n+1}}} \cdot \boxed{\frac{\partial \boldsymbol{a}_{n+1}}{\partial W_n}}$$

dadxProductNext

$$\text{dadxProduct} = \text{dadxProductNext} \cdot \frac{\partial \boldsymbol{a}_{n+1}}{\partial \boldsymbol{a}_n}$$

各模块功能与实现

三、网络层模块（Layers）

反向传播的实现

其它各个具体的网络层只需要继承此基础网络层，并根据上面的前向计算与反向传播的规则进行定义即可。

根据上式（链式法则），损失函数层（最后一层）的dadxProduct应当定义为 $\frac{\partial J}{\partial \boldsymbol{a}_N}$，其中$J$为性能函数。

各模块功能与实现

三、网络层模块（Layers）

反向传播的实现

举例来说，对于一个卷积层，本层完成的计算为：

$$a_{n,m}^{l+1} = \sum_{i=1}^{I}\sum_{j=1}^{J} a_{n+j-1,m+i-1}^{l} \cdot w_{j,i}^{l} + b_{n,m}$$

则每层只需要计算：

$$\frac{\partial a_{n,m}^{l+1}}{\partial a_{x,y}^{l}} = \begin{cases} w_{x-n+1,y-m+1}^{l}, \text{ if } x-n+1 < filter_size \text{ and } y-m+1 < filter_size \\ 0, \text{ elsewhere} \end{cases}$$

$$\frac{\partial a_{n,m}^{l+1}}{\partial w_{j,i}^{l}} = a_{n+j-1,m+i-1}^{l} \qquad \frac{\partial a_{n,m}^{l+1}}{\partial b_{n,m}} = 1$$

各模块功能与实现

三、网络层模块（Layers）

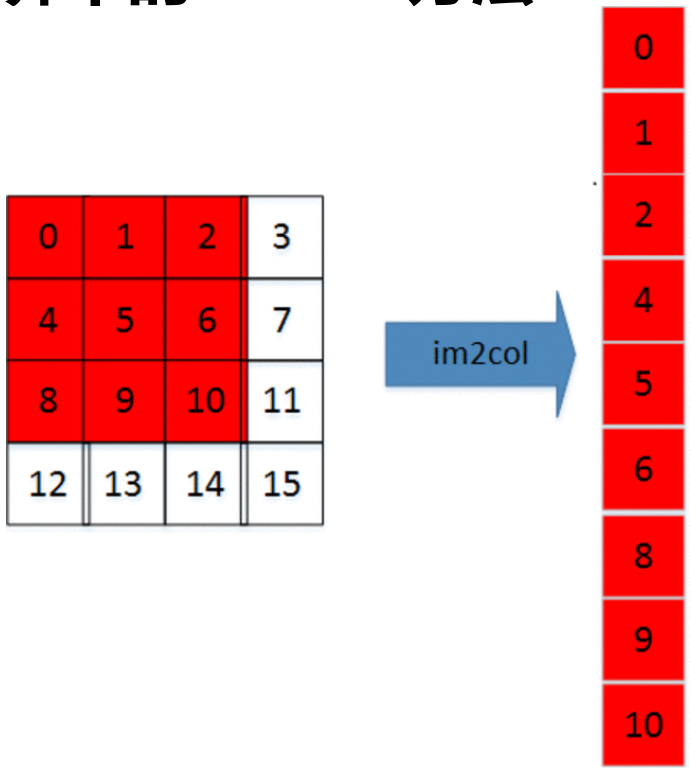
卷积运算中的im2col方法

im2col的核心思想是将输入图像的每次卷积的视野与卷积核感受野部分转化成一行（列）来存储，优化运算速度，减少内存访问时间。

各模块功能与实现

三、网络层模块（Layers）

caffe卷积运算中的im2col方法



各模块功能与实现

四、优化算法模块（Optimizer）

该模块在基础优化算法类Optimizer类的基础上继承定义了多个优化算法类，例如SGD优化算法类、Adam优化算法类、Momentum优化算法类、RMSProp优化算法类等等。

```
class BaseOptimizer:
    def __init__(self, learning_rate):
        self.learning_rate = learning_rate
    def update(self, weights, grads):
        for k in weights.keys():
            weights[k] -= self.learning_rate * grads[k]
```

各模块功能与实现

四、优化算法模块（Optimizer）

（1）SGD算法

其参数更新方式为：

$$\begin{cases} v_t = -\eta \nabla_\theta J(\theta_t) \\ \theta_{t+1} = \theta_t + v_t \end{cases}$$

其中，η代表学习率，θ_t代表t时刻的参数，$\nabla_\theta J(\theta_t)$代表参数$\theta$在$t$时刻的导数，$v_t$ 代表参数的更新速度。

各模块功能与实现

四、优化算法模块（Optimizer）

（2）Momentum算法

其参数更新方式为：

$$\begin{cases} v_t = -\eta \nabla_\theta J(\theta_t) + \alpha v_{t-1} \\ \theta_{t+1} = \theta_t + v_t \end{cases}$$

其中，η代表学习率，θ_t代表t时刻的参数，$\nabla_\theta J(\theta_t)$代表参数$\theta$在$t$时刻的导数，$v_t$ 代表参数的更新速度，α一般取0.9。

各模块功能与实现

四、优化算法模块（Optimizer）

（3）Adam算法

其参数更新方式为：

各模块功能与实现

四、优化算法模块（Optimizer）

$$
\begin{cases}
g_t = \nabla_\theta J(\theta_t) \\
m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t = \dfrac{m_t}{1-\beta_1} \\
\hat{v}_t = \dfrac{v_t}{1-\beta_2} \\
\theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \hat{m}_t
\end{cases}
$$

其中，t 表示 t 时刻，m_t 、v_t 分别是梯度的一阶矩估计和二阶矩估计，$\hat{m}_t$、$\hat{v}_t$ 分别是对 m_t 、v_t 的校正，这样可以近似为对期望的无偏估计。

各模块功能与实现

五、训练器模块（Trainer）

该模块定义了一个训练器类，其中实现了网络的参数更新、性能指标计算与日志输出等等操作。该类的实现就是利用已经定义好的工具，如优化器、网络模型等等，执行网络的训练流程。

各模块功能与实现

六、基础网络模块（Base Net）

该模块定义了一个最基础、最简单的网络模型，将前向计算、梯度计算、损失函数输出计算、准确率计算、模型读取与保存等等各个模型共有的功能进行了封装，并保证了高通用性。

各模块功能与实现

六、基础网络模块（Base Net)

```python
class BaseNet:
    def __init__(self, input_size, hidden_size, output_size):
        # weights dictionary
        self.weights = {}
        self.input_size = input_size
        self.output_size = output_size
        # layers dictionary
        self.layers = OrderedDict()
        self.loss_function_layer = None
        self.hidden_size = hidden_size

    def forward(self, x):
        for layer in self.layers.values():
            x = layer.forward(x)
        return x
```

```python
    def loss(self, x, y):
        a = self.forward(x)
        return self.loss_function_layer.forward(a, y)

    def accuracy(self, x, y, batch_size=100):
        acc = 0.0
        for i in range(int(x.shape[0] / batch_size)):
            # avoid too large input
            x_bat = x[i*batch_size:(i+1)*batch_size]
            y_bat = y[i*batch_size:(i+1)*batch_size]
            a = self.forward(x_bat)
            a = np.argmax(a, axis=1)
            acc += np.sum(a == y_bat)
        return acc / x.shape[0]
```

各模块功能与实现

六、基础网络模块（Base Net)

```
def gradient(self, x, y):
    # forward computation
    self.loss(x, y)
    # backward propagation
    dadxProduct = 1
    dadxProduct = self.loss_function_layer.backward(dadxProduct)

    layers = list(self.layers.values())
    layers.reverse()
    for layer in layers:
        dadxProduct = layer.backward(dadxProduct)

    grads = {}
    i = 1
    for k in self.layers:
        if not(np.array(self.layers[k].gradW).any()==None):
            grads['W'+str(i)], grads['b'+str(i)] = self.layers[k].gradW, self.layers[k].gradb
            i += 1
    return grads
```

各模块功能与实现

六、基础网络模块（Base Net）

```python
def save_model(self, file_name="model.pkl"):
    model = {}
    for key, val in self.weights.items():
        model[key] = val
    with open(file_name, 'wb') as f:
        pickle.dump(model, f)
```

```python
def load_model(self, file_name="model.pkl"):
    with open(file_name, 'rb') as f:
        model = pickle.load(f)
    for key, val in model.items():
        self.weights[key] = val
    for i, key in enumerate(['Conv1', 'Affine1', 'Affine2']):
        self.layers[key].W = self.weights['W' + str(i+1)]
        self.layers[key].b = self.weights['b' + str(i+1)]
```

各模块功能与实现

六、基础网络模块（Base Net)

感知机学习规则为（向量形式）：

$$W^{new} = W^{old} + (\boldsymbol{t} - \boldsymbol{a}) p^T$$

$$\boldsymbol{b}^{new} = \boldsymbol{b}^{old} + (\boldsymbol{t} - \boldsymbol{a})$$

其中 $\boldsymbol{t}$ 为标签，$\boldsymbol{a}$ 为网络预测。

Hebb学习规则为（向量形式）：

$$W^{new} = W^{old} + \boldsymbol{t}_q \boldsymbol{p}_q^T$$

其中 $\boldsymbol{t}$ 为标签，$\boldsymbol{p}$ 为网络输入（样本）。

各模块功能与实现

七、性能指标模块（Performance）

该模块根据混淆矩阵定义实现了混淆矩阵的计算与绘制，只需要提供预测值和标签即可计算，并提供了根据混淆矩阵求出精确率（Precision）、召回率（Recall）、F1值（F1-score）的函数，便于在网络训练时和网络训练后进行调用，查看这几项性能指标，并相应地进行网络的调整与优化。

实验与分析

设计6个测试实验：

① 感知机学习算法测试；
② Hebb学习算法测试之实现自联想存储器；
③ 卷积神经网络搭建与测试之MNIST分类；
④ 全连接神经网络搭建与测试之MNIST分类；
⑤ 新网络层的定义测试之Cora数据集上图卷积层搭建测试；
⑥ 二维复杂曲线拟合。

实验与分析

① 感知机学习算法测试实验

$$train_x = \left\{ \begin{bmatrix} 1 \\ 1 \\ 3 \end{bmatrix} \quad \begin{bmatrix} 3 \\ 3 \\ 6 \end{bmatrix} \quad \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \right\} \qquad train_y = \left\{ \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\}$$

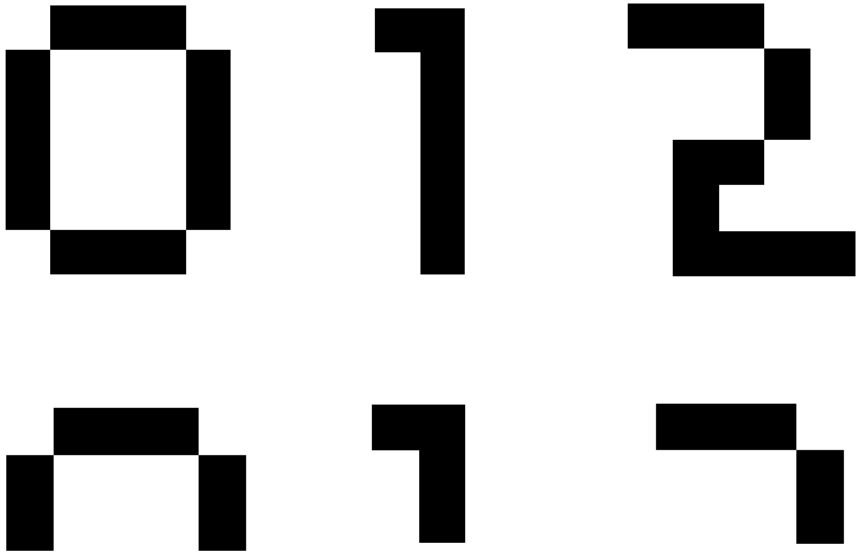
实验与分析

① 感知机学习算法测试实验结果与分析

```
Training finished! (All the samples are correctly classified.)

分类测试:
--------------------
Predict:
 [[0.]
 [1.]]
True:
 [[0]
 [1]]

Correct!
--------------------
Predict:
 [[1.]
 [1.]]
True:
 [[1]
 [1]]

Correct!
--------------------
Predict:
 [[0.]
 [0.]]
True:
 [[0]
 [0]]

Correct!
--------------------
```

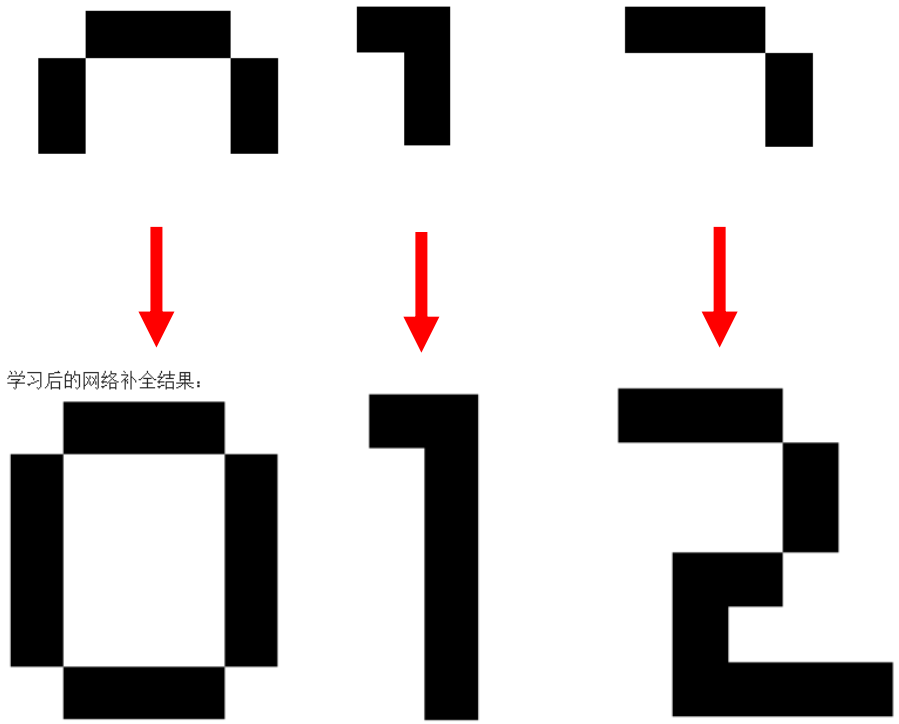
实验与分析

② Hebb学习算法测试之实现自联想存储器实验



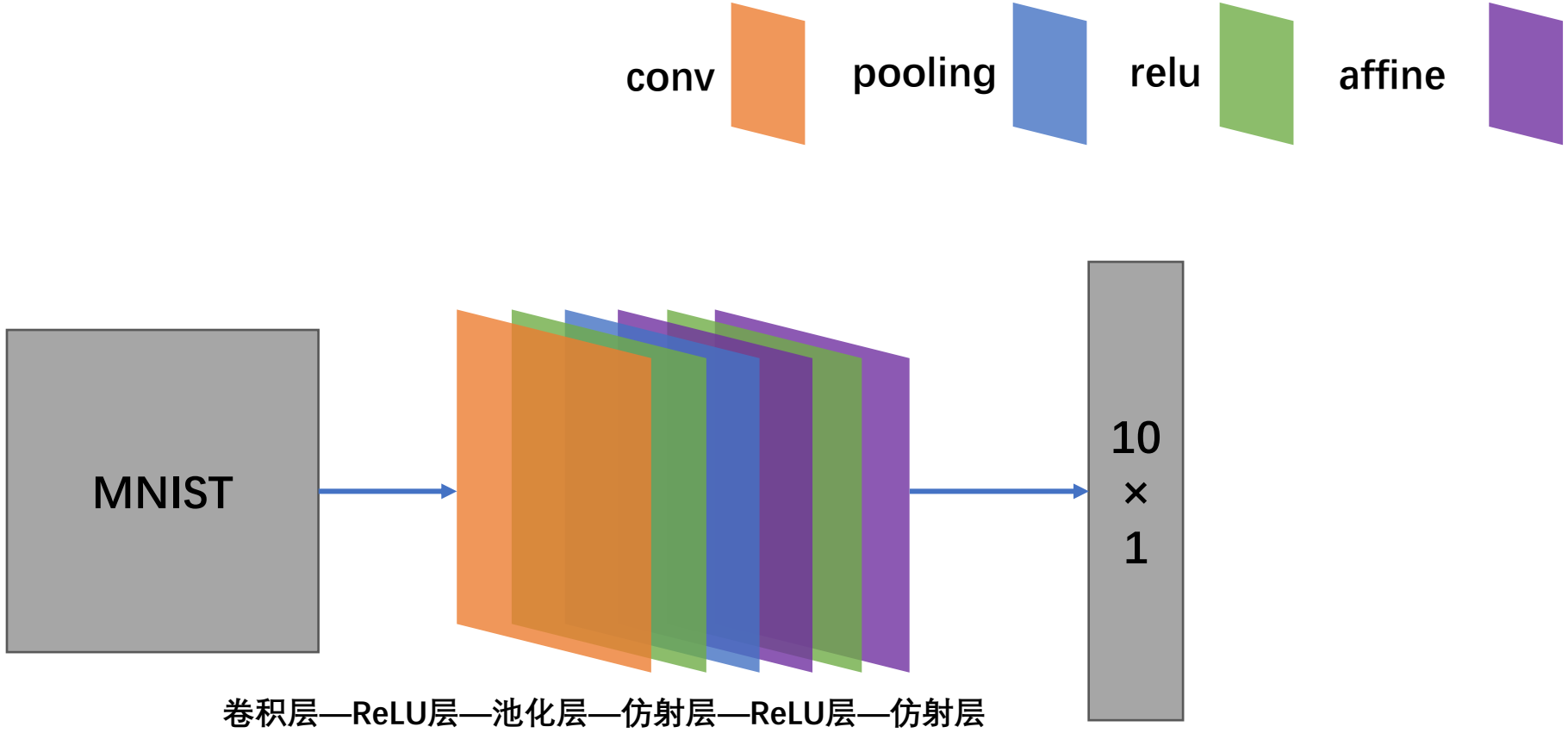
实验与分析

② Hebb学习算法测试之实现自联想存储器实验结果与分析



实验与分析

③ 卷积神经网络搭建与测试之MNIST分类实验



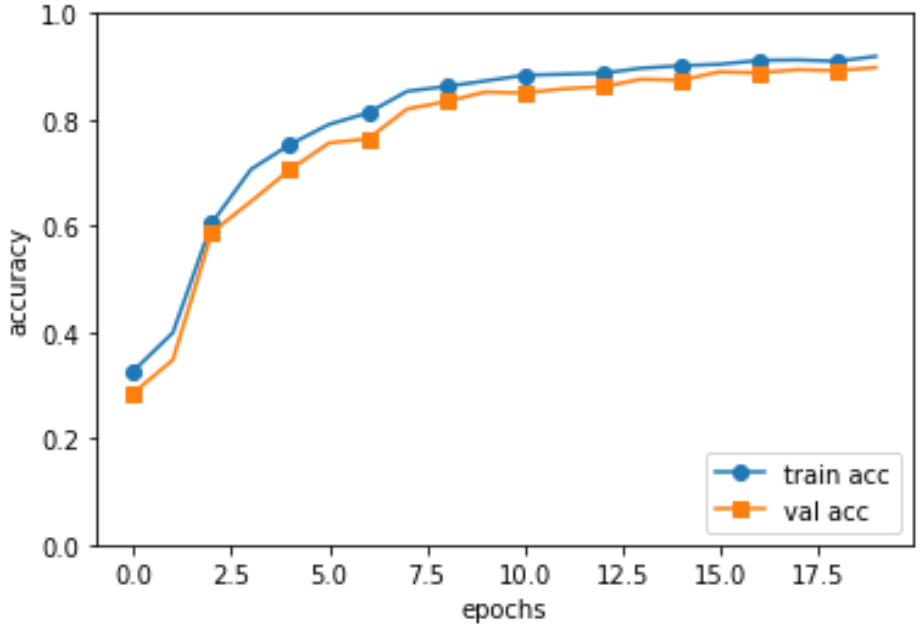
实验与分析

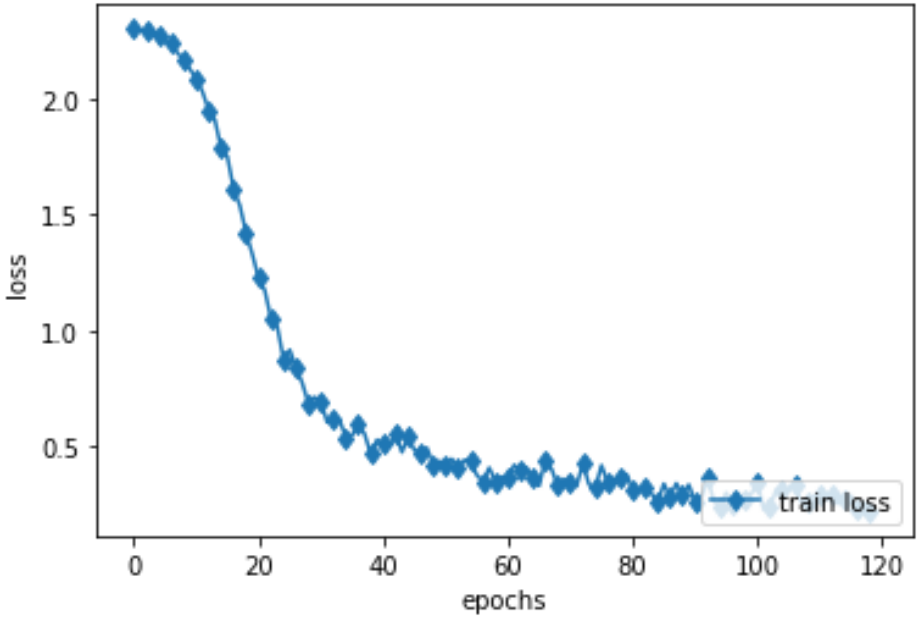
③ 卷积神经网络搭建与测试之MNIST分类实验结果与分析

```
Epoch 1: Loss 2.3001996003018004, TrainAcc 0.21, ValAcc 0.25
Epoch 2: Loss 2.240442108239152, TrainAcc 0.354, ValAcc 0.364
Epoch 3: Loss 2.0050426200366385, TrainAcc 0.5326666666666666, ValAcc 0.51
Epoch 4: Loss 1.4928457506869692, TrainAcc 0.6976666666666667, ValAcc 0.626
Epoch 5: Loss 0.933063103253369, TrainAcc 0.767, ValAcc 0.73
Epoch 6: Loss 0.65065897906747, TrainAcc 0.7983333333333333, ValAcc 0.754
Epoch 7: Loss 0.572772244501228, TrainAcc 0.8246666666666667, ValAcc 0.786
Epoch 8: Loss 0.4736580910837395, TrainAcc 0.8476666666666667, ValAcc 0.81
Epoch 9: Loss 0.3873480964968936, TrainAcc 0.8606666666666667, ValAcc 0.836
Epoch 10: Loss 0.4158817819343124, TrainAcc 0.8793333333333333, ValAcc 0.844
Epoch 11: Loss 0.3419276144386179, TrainAcc 0.888, ValAcc 0.868
Epoch 12: Loss 0.4299549363968425, TrainAcc 0.8896666666666667, ValAcc 0.878
Epoch 13: Loss 0.30929469511982816, TrainAcc 0.897, ValAcc 0.87
Epoch 14: Loss 0.26274307033175703, TrainAcc 0.9016666666666666, ValAcc 0.88
Epoch 15: Loss 0.3145289170421933, TrainAcc 0.909, ValAcc 0.882
Epoch 16: Loss 0.2559214266105579, TrainAcc 0.913, ValAcc 0.898
Epoch 17: Loss 0.2920241090402168, TrainAcc 0.916, ValAcc 0.904
Epoch 18: Loss 0.3236169070289904, TrainAcc 0.925, ValAcc 0.906
Epoch 19: Loss 0.23426512980603703, TrainAcc 0.9246666666666666, ValAcc 0.902
Epoch 20: Loss 0.2588528594548765, TrainAcc 0.9293333333333333, ValAcc 0.9
Training finished! Final test accuracy is 0.882
Model successfully saved, congratulations!
```

实验与分析

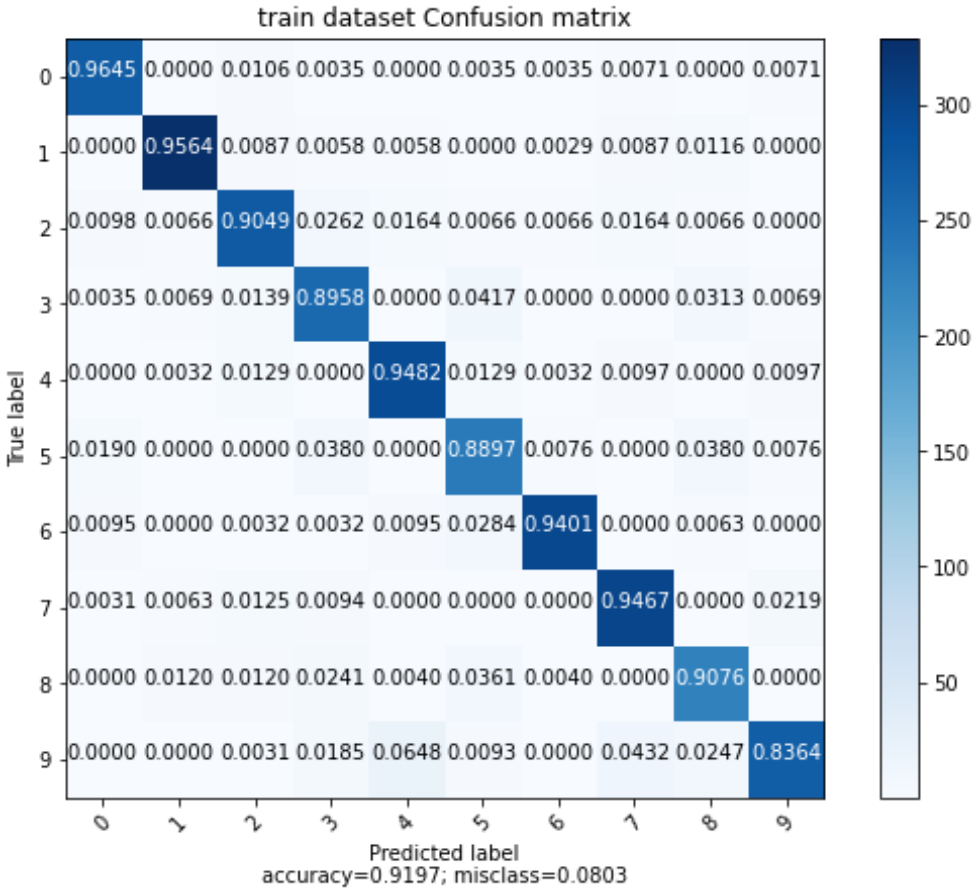
③ 卷积神经网络搭建与测试之MNIST分类实验结果与分析

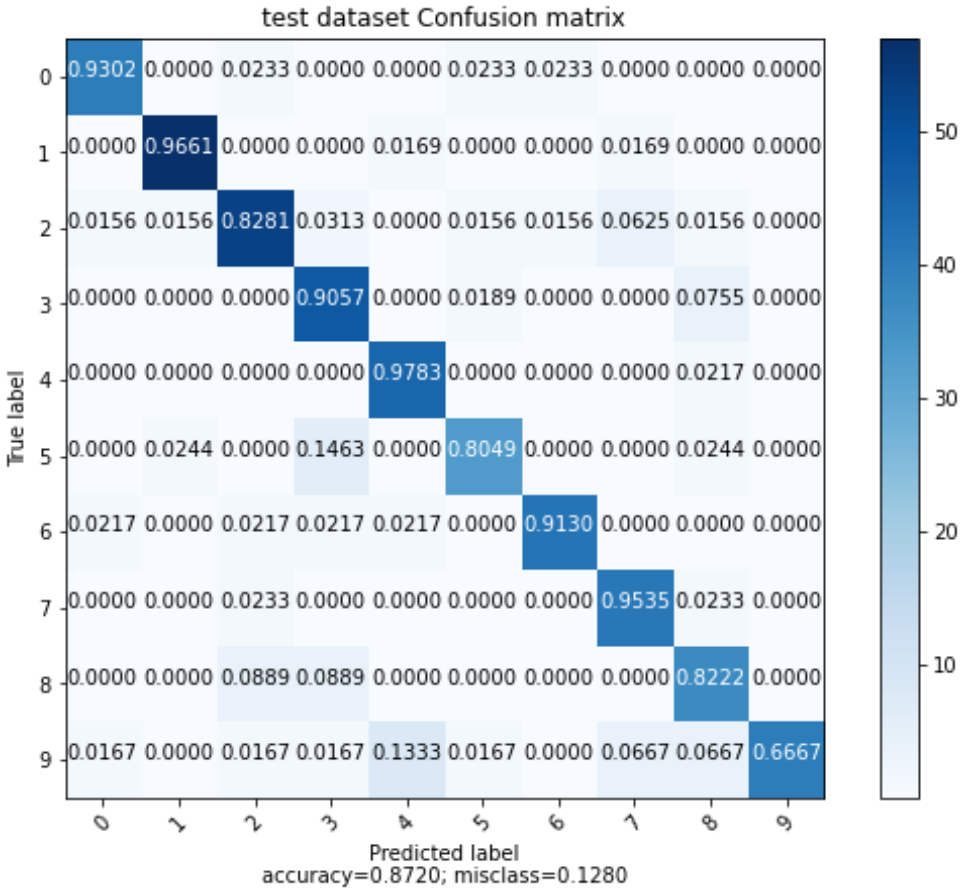




实验与分析

③ 卷积神经网络搭建与测试之MNIST分类实验结果与分析





实验与分析

③ 卷积神经网络搭建与测试之MNIST分类实验结果与分析

```
(train set) Precision, Recall and F1-score:
Class 0:
precision: 0.972        recall: 0.962   f1-score: 0.967
Class 1:
precision: 0.976        recall: 0.940   f1-score: 0.958
Class 2:
precision: 0.926        recall: 0.902   f1-score: 0.914
Class 3:
precision: 0.864        recall: 0.948   f1-score: 0.904
Class 4:
precision: 0.923        recall: 0.955   f1-score: 0.939
Class 5:
precision: 0.894        recall: 0.881   f1-score: 0.888
Class 6:
precision: 0.971        recall: 0.961   f1-score: 0.966
Class 7:
precision: 0.936        recall: 0.931   f1-score: 0.933
Class 8:
precision: 0.900        recall: 0.900   f1-score: 0.900
Class 9:
precision: 0.909        recall: 0.897   f1-score: 0.903
```

```
(test set) Precision, Recall and F1-score:
Class 0:
precision: 0.977        recall: 0.933   f1-score: 0.955
Class 1:
precision: 0.966        recall: 0.966   f1-score: 0.966
Class 2:
precision: 0.885        recall: 0.844   f1-score: 0.864
Class 3:
precision: 0.758        recall: 0.922   f1-score: 0.832
Class 4:
precision: 0.818        recall: 0.978   f1-score: 0.891
Class 5:
precision: 0.865        recall: 0.800   f1-score: 0.831
Class 6:
precision: 0.955        recall: 0.955   f1-score: 0.955
Class 7:
precision: 0.860        recall: 0.915   f1-score: 0.887
Class 8:
precision: 0.837        recall: 0.804   f1-score: 0.820
Class 9:
precision: 0.950        recall: 0.717   f1-score: 0.817
```

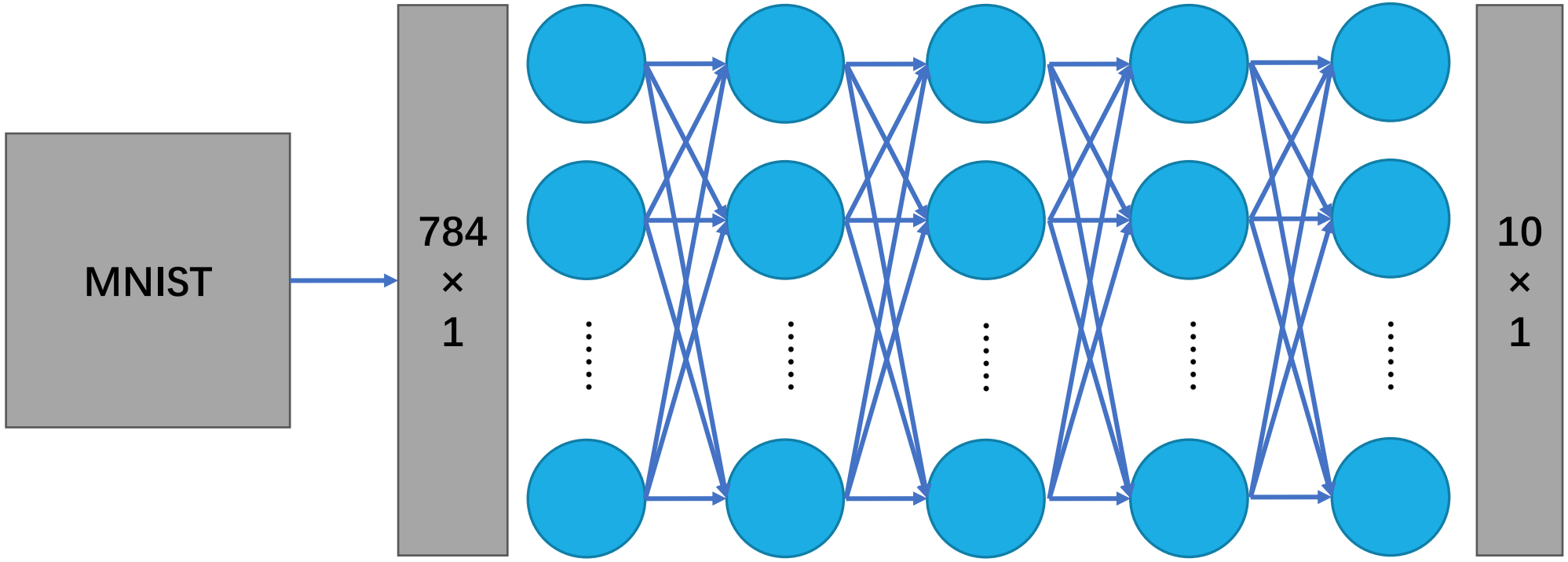
实验与分析

③ 卷积神经网络搭建与测试之MNIST分类实验结果与分析

```
Predict: 3
True: 3
Correct?  True
-------------------------------
Predict: 9
True: 9
Correct?  True
-------------------------------
Predict: 3
True: 5
Correct?  False
-------------------------------
```

实验与分析

④ 全连接神经网络搭建与测试之MNIST分类实验



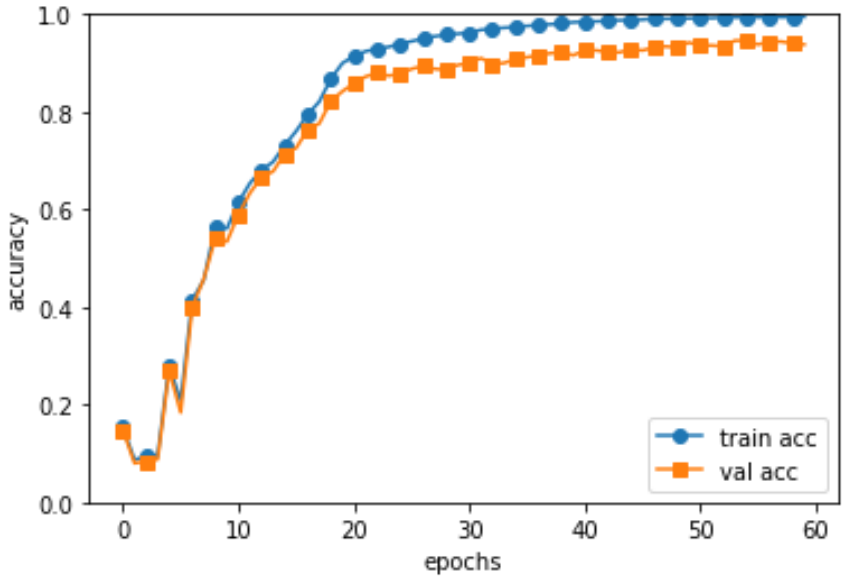
实验与分析

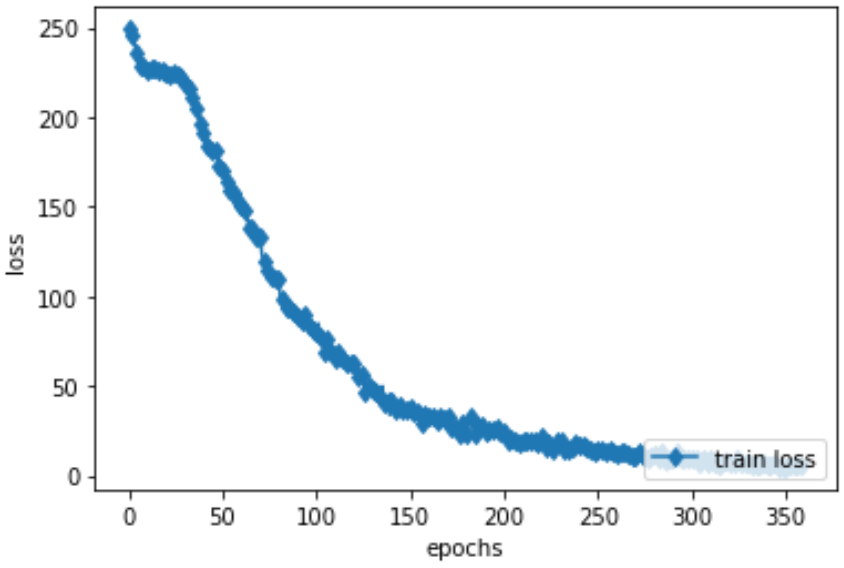
④ 全连接神经网络搭建与测试之MNIST分类实验结果与分析

```
Epoch 52: Loss 11.409583577405598, TrainAcc 0.9893333333333333, ValAcc 0.912
Epoch 53: Loss 11.719927098227664, TrainAcc 0.9893333333333333, ValAcc 0.908
Epoch 54: Loss 8.17334867981969, TrainAcc 0.9896666666666667, ValAcc 0.91
Epoch 55: Loss 9.9331465517391, TrainAcc 0.9906666666666667, ValAcc 0.91
Epoch 56: Loss 7.508938174392356, TrainAcc 0.9903333333333333, ValAcc 0.916
Epoch 57: Loss 10.763867457750138, TrainAcc 0.991, ValAcc 0.916
Epoch 58: Loss 6.912677907236992, TrainAcc 0.991, ValAcc 0.904
Epoch 59: Loss 9.108005145295916, TrainAcc 0.9913333333333333, ValAcc 0.916
Epoch 60: Loss 6.918187815145909, TrainAcc 0.9926666666666667, ValAcc 0.912
Training finished! Final test accuracy is 0.896
Model successfully saved, congratulations!
```

实验与分析

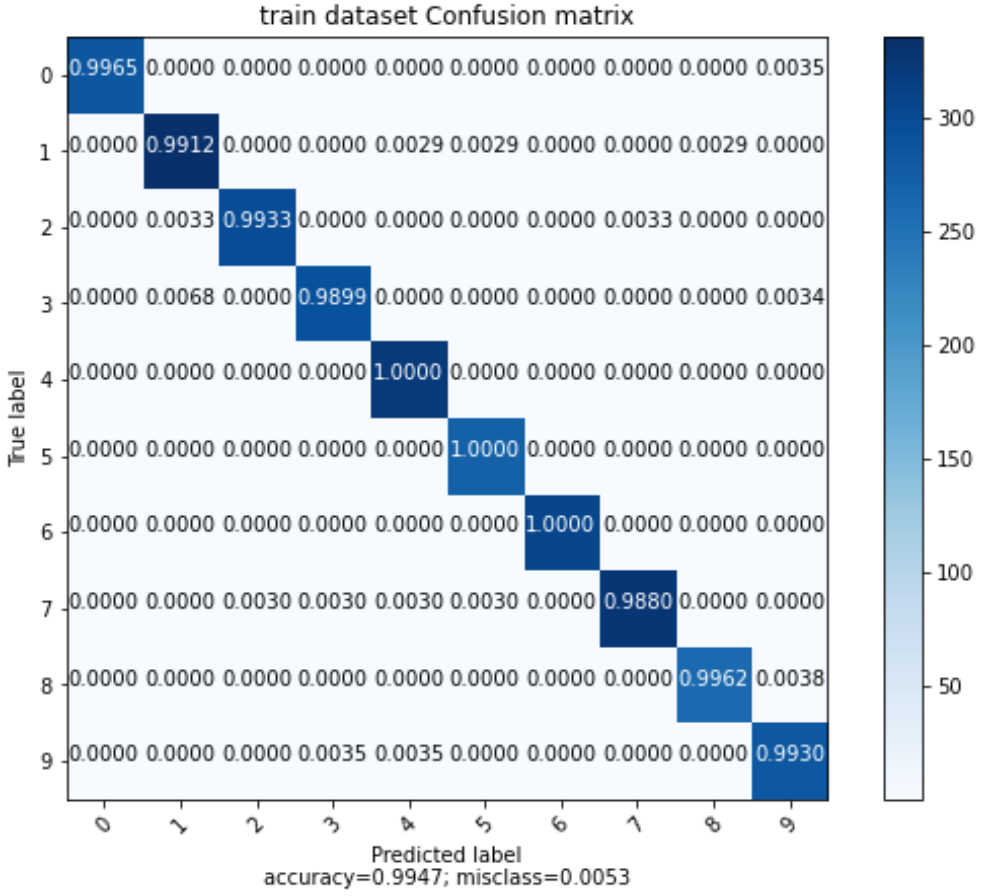
④ 全连接神经网络搭建与测试之MNIST分类实验结果与分析

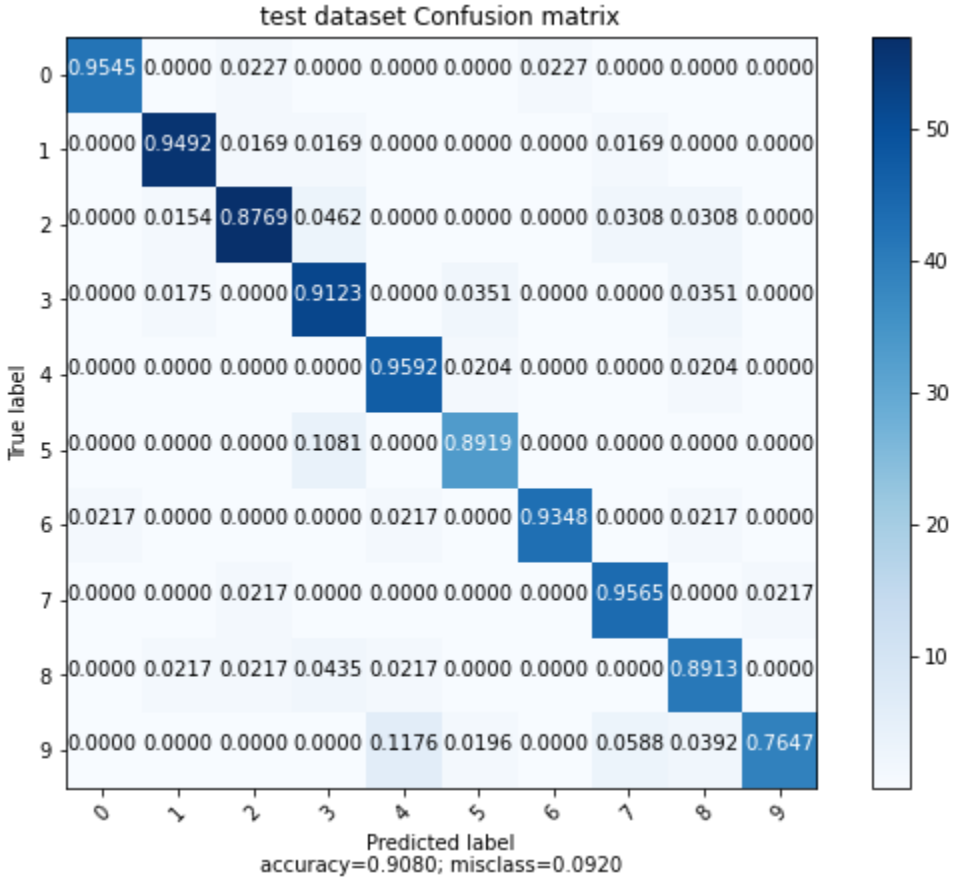




实验与分析

④ 全连接神经网络搭建与测试之MNIST分类实验结果与分析





实验与分析

④ 全连接神经网络搭建与测试之MNIST分类实验结果与分析

```
(train set) Precision, Recall and F1-score:
Class 0:
precision: 0.996        recall: 0.996   f1-score: 0.996
Class 1:
precision: 0.988        recall: 0.997   f1-score: 0.993
Class 2:
precision: 0.997        recall: 0.990   f1-score: 0.993
Class 3:
precision: 0.990        recall: 0.990   f1-score: 0.990
Class 4:
precision: 0.991        recall: 1.000   f1-score: 0.995
Class 5:
precision: 0.985        recall: 1.000   f1-score: 0.993
Class 6:
precision: 1.000        recall: 0.994   f1-score: 0.997
Class 7:
precision: 0.985        recall: 0.991   f1-score: 0.988
Class 8:
precision: 1.000        recall: 0.985   f1-score: 0.992
Class 9:
precision: 0.990        recall: 0.976   f1-score: 0.983
```

```
(test set) Precision, Recall and F1-score:
Class 0:
precision: 0.977        recall: 0.977   f1-score: 0.977
Class 1:
precision: 0.949        recall: 0.982   f1-score: 0.966
Class 2:
precision: 0.902        recall: 0.846   f1-score: 0.873
Class 3:
precision: 0.823        recall: 0.911   f1-score: 0.864
Class 4:
precision: 0.873        recall: 0.960   f1-score: 0.914
Class 5:
precision: 0.919        recall: 0.872   f1-score: 0.895
Class 6:
precision: 0.955        recall: 0.933   f1-score: 0.944
Class 7:
precision: 0.820        recall: 0.932   f1-score: 0.872
Class 8:
precision: 0.837        recall: 0.820   f1-score: 0.828
Class 9:
precision: 0.950        recall: 0.745   f1-score: 0.835
```

实验与分析

④ 全连接神经网络搭建与测试之MNIST分类实验结果与分析

```
Predict: 3
True: 3
Correct?  True
--------------------------------
Predict: 9
True: 9
Correct?  True
--------------------------------
Predict: 5
True: 5
Correct?  True
--------------------------------
```

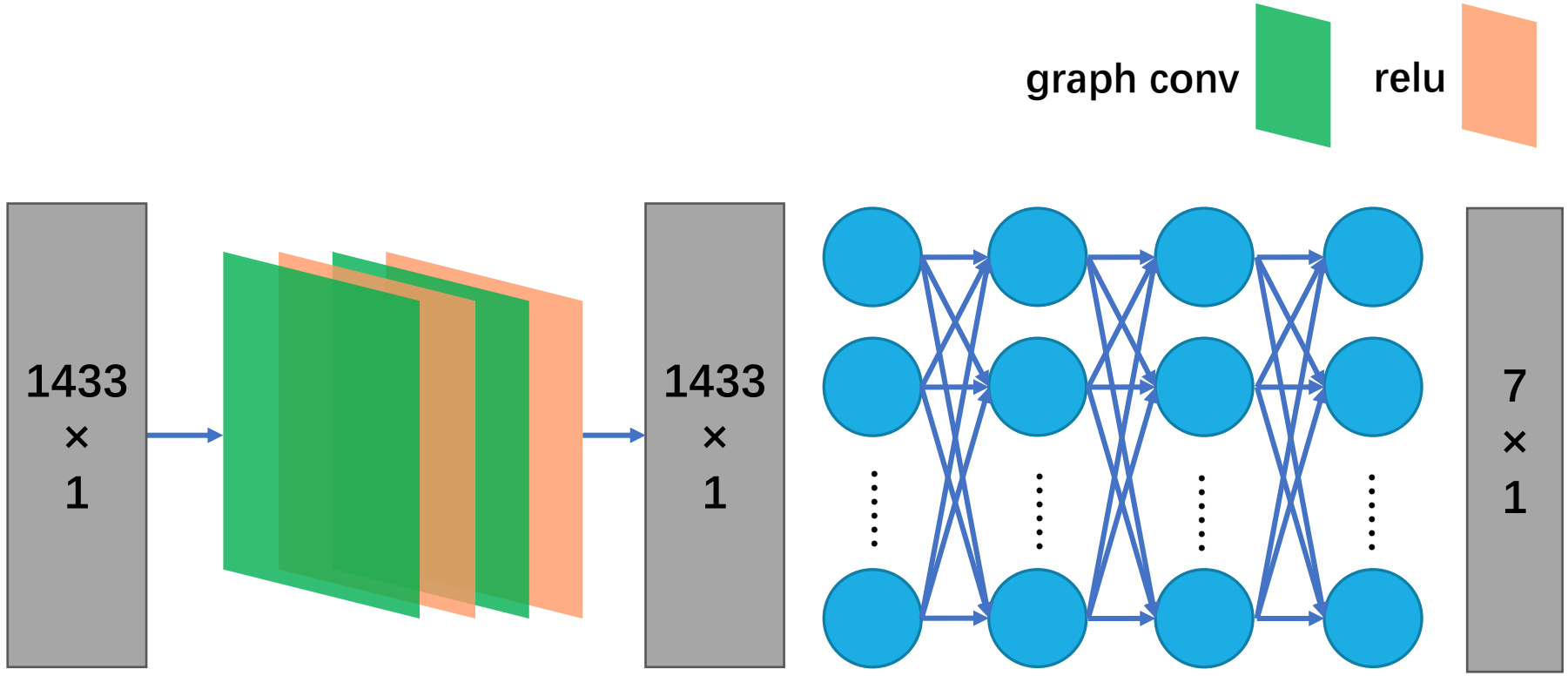
实验与分析

⑤ 新网络层的定义测试之Cora数据集上图卷积层搭建测试实验

```python
# 图卷积层
class GraphConvolutionLayer(BaseLayer):
    def __init__(self, A):
        super(GraphConvolutionLayer, self).__init__(None, None, None, None)
        self.A = A
    def forward(self, x):
        self.x = x
        a = self.A.dot(x)
        return a
```

实验与分析

⑤ 新网络层的定义测试之Cora数据集上图卷积层搭建测试实验



图卷积层—ReLU层—图卷积层—ReLU层

1433, 100, 50, 7

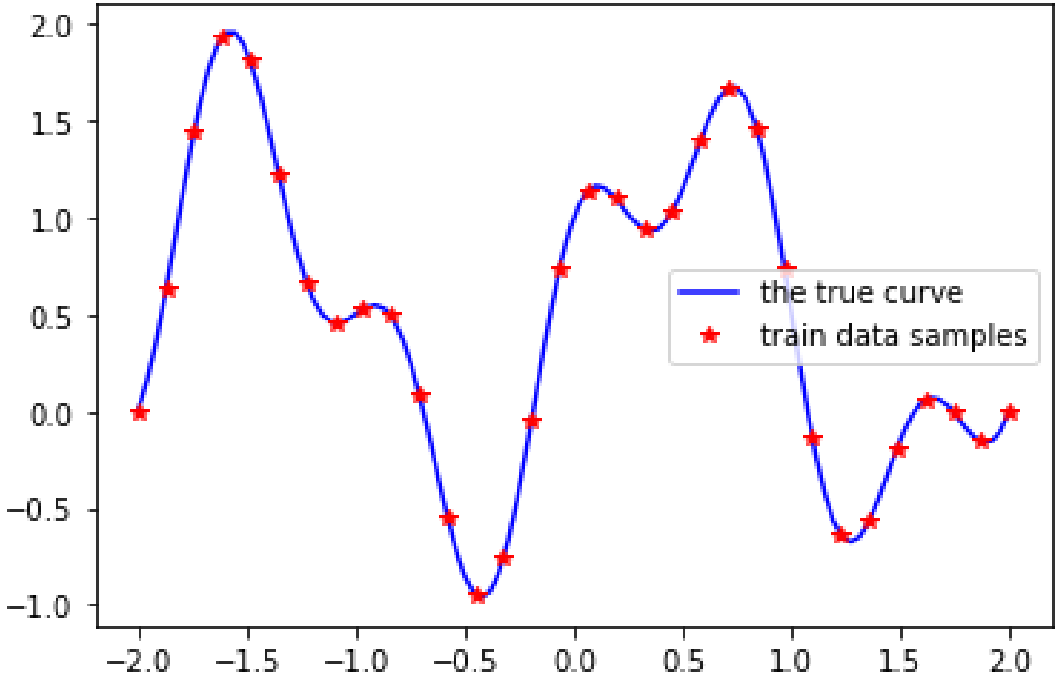
实验与分析

⑤ 新网络层的定义测试之Cora数据集上图卷积层搭建测试实验结果与分析

```
Epoch 47: Loss 1.1173286568964353, TrainAcc 0.5285714285714286, ValAcc 0.616
Epoch 48: Loss 1.135967799689345, TrainAcc 0.5357142857142857, ValAcc 0.612
Epoch 49: Loss 1.1260773132412436, TrainAcc 0.5357142857142857, ValAcc 0.608
Epoch 50: Loss 1.1354043145768686, TrainAcc 0.5357142857142857, ValAcc 0.608
Epoch 51: Loss 1.1204326786857892, TrainAcc 0.5357142857142857, ValAcc 0.604
Epoch 52: Loss 1.0983099648990229, TrainAcc 0.5357142857142857, ValAcc 0.602
Epoch 53: Loss 1.0727567346232323, TrainAcc 0.5357142857142857, ValAcc 0.598
Epoch 54: Loss 1.0780614551881642, TrainAcc 0.5357142857142857, ValAcc 0.592
Epoch 55: Loss 1.1322274126675365, TrainAcc 0.5357142857142857, ValAcc 0.602
Epoch 56: Loss 1.0932339883092888, TrainAcc 0.5357142857142857, ValAcc 0.6
Epoch 57: Loss 1.1299857553433832, TrainAcc 0.5357142857142857, ValAcc 0.604
Epoch 58: Loss 1.0899884053864553, TrainAcc 0.5357142857142857, ValAcc 0.602
Epoch 59: Loss 1.1693748824886128, TrainAcc 0.5357142857142857, ValAcc 0.604
Epoch 60: Loss 1.0585679257157135, TrainAcc 0.5357142857142857, ValAcc 0.608
Training finished! Final test accuracy is 0.57
Model successfully saved, congratulations!
```

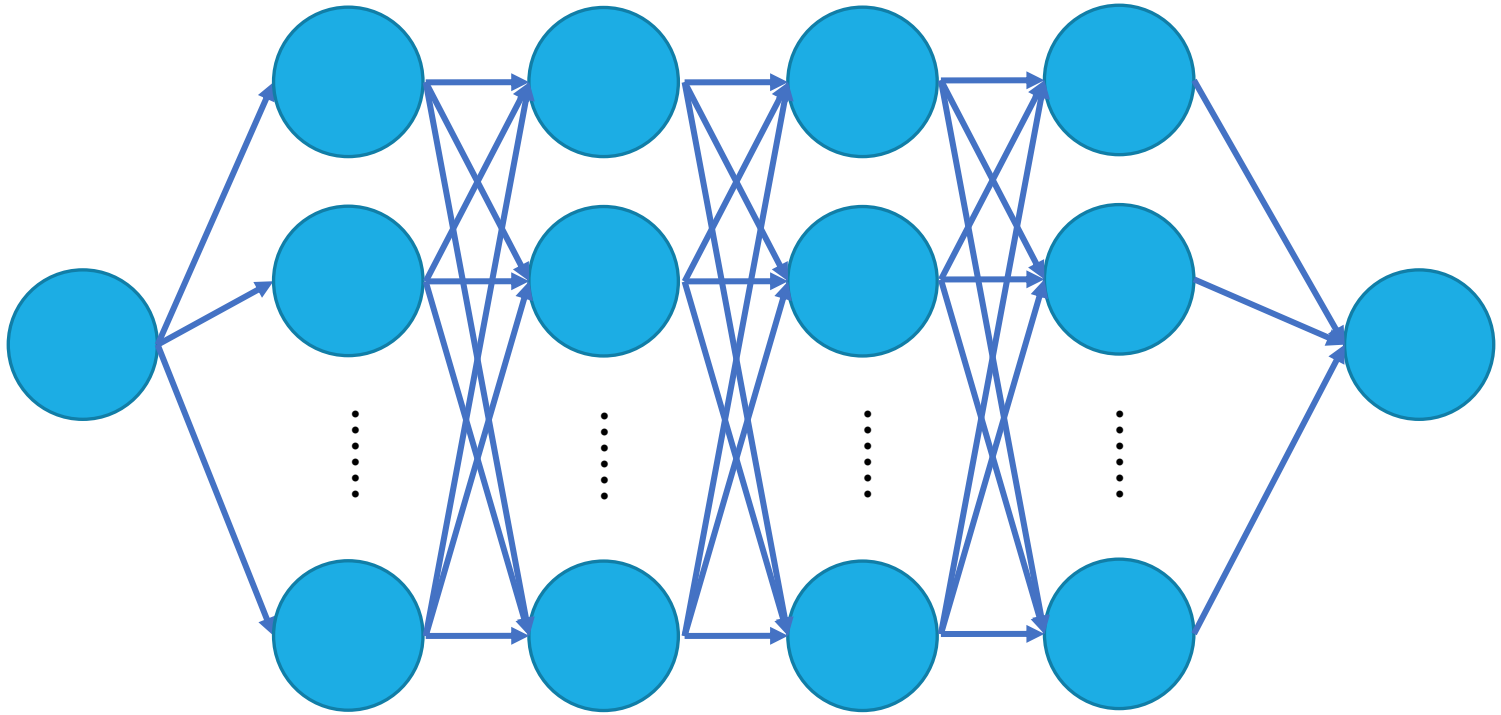
实验与分析

⑥ 二维复杂曲线拟合实验



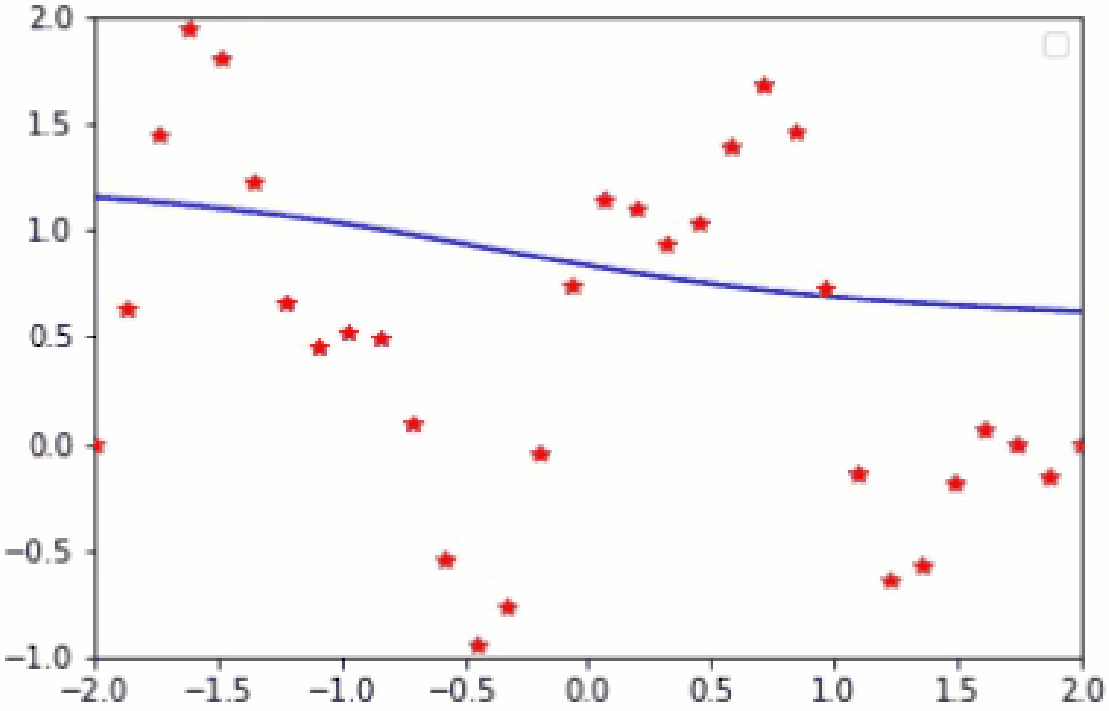
实验与分析

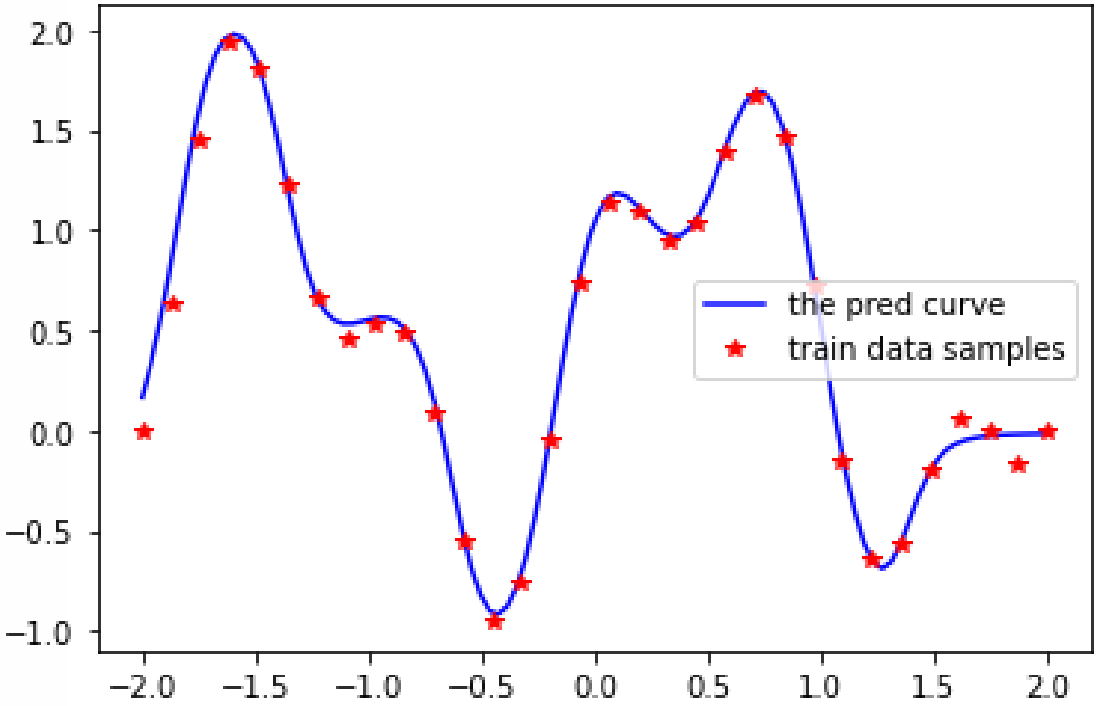
⑥ 二维复杂曲线拟合实验



实验与分析

⑥ 二维复杂曲线拟合实验结果与分析





PS

The sample codes and handbook of the sample framework are available at
https://github.com/XuChentianye/Design_Your_Own_Deep_Learning_Framework